Image-based crop disease detection with federated learning

Summer Internship Report

Submitted by:
Angajala Dinesh Kartheek
Roll No : 23CS01005

Under the Guidance of:
Dr. Barathram Ramkumar
Associate Professor
School of Electrical and Computer Sciences (SECS)
Indian Institute of Technology Bhubaneswar

1. INTRODUCTION

1.1 Background

Crop diseases can reduce agricultural productivity and crop quality. Image-based deep learning has become a useful approach for automatically identifying disease patterns from plant leaf images. The reference study motivating this internship investigates crop disease classification using federated learning, where multiple participating learners train local models and a central server aggregates their updates without collecting raw training images at the server.

Federated learning is a distributed machine learning paradigm. A global model is initialized and sent to participating clients. Each client performs local optimization using its own data, returns model parameters or updates, and the server aggregates the received models. The aggregated global model is then redistributed for the next communication round. The reference paper describes this workflow using five phases: initialization, local training, parameter transfer, aggregation, and global model transfer/evaluation.

1.2 Motivation

A centralized crop-disease classifier can require the collection and transfer of large image datasets. Federated learning offers an alternative experimental framework in which local datasets remain within their assigned clients while model parameters are aggregated. The practical motivation for this internship was therefore to implement a complete federated training pipeline and study whether commonly used CNN architectures can obtain high disease-classification performance under repeated federated aggregation.

1.3 Project Scope

The final project scope intentionally focuses on three CNN architectures - ResNet50, VGG16, and MobileNetV2 - and four PlantVillage crop datasets - Grape, Apple, Corn, and Tomato. The final working configuration was fixed at three clients, one local epoch, and 30 communication rounds. Thus, 3 x 4 = 12 experiments were performed. The broader reference paper also contains parameter studies over client count, communication rounds, and local epochs; those additional variations were not included in the final internship experiment set.

2. BACKGROUND AND LITERATURE REVIEW

2.1 Reference Study

The principal reference for this project is Denis Mamba Kabala, Adel Hafiane, Laurent Bobelin, and Raphaël Canals, "Image-based crop disease detection with federated learning", Scientific Reports, 13, 19220 (2023), DOI: 10.1038/s41598-023-46218-5. The paper examines federated learning for crop disease classification using PlantVillage data and evaluates multiple CNN and vision-transformer architectures.

2.2 Federated Learning

The reference paper describes a client-server learning cycle. Each client receives the global model, trains locally, and returns its updated parameters. The server aggregates the local models

to produce a new global model. A key advantage of this architecture is that raw training data need not be transferred to the central server. The paper uses Federated Averaging (FedAvg) as its aggregation method, in which client models are averaged with weights proportional to their local sample counts. Figure 1 summarizes one communication round of this cycle as implemented in this project.

2.3 Relation of the Internship Work to the Paper

The internship implementation follows the central ideas of the paper: PlantVillage crop subsets, pretrained CNN architectures, simulated multiple clients, local training, weighted FedAvg aggregation, repeated communication rounds, and evaluation using Accuracy and F1-score. The project narrows the model set to ResNet50, VGG16, and MobileNetV2 and uses a fixed configuration of three clients, one local epoch, and 30 rounds for all 12 final experiments.

3. PROBLEM STATEMENT AND OBJECTIVES

3.1 Problem Statement

Develop and evaluate a federated deep learning pipeline for multi-class plant leaf disease classification on selected PlantVillage crop datasets, while keeping the client datasets logically decentralized and using a common global model obtained through FedAvg aggregation.

3.2 Objectives

- Implement a reproducible federated learning workflow using PyTorch.
- Use three pretrained CNN architectures: ResNet50, VGG16, and MobileNetV2.
- Evaluate the models on four crop datasets: Grape, Apple, Corn, and Tomato.
- Simulate three participating clients with equal random data partitioning.
- Train for one local epoch per client in each of 30 communication rounds.
- Aggregate client models with weighted Federated Averaging.

- Measure Accuracy and weighted F1-score after every communication round.
- Compare final model performance and execution time across all 12 experiments.
- Record limitations and identify future improvements for a more complete federated evaluation.

4. METHODOLOGY

4.1 Overall Workflow

The implementation follows the sequence below:

- Load the selected crop dataset from the train and validation folders.
- Check that the expected class folders exist and that train/validation class labels match.
- Compute RGB mean and standard deviation from the training images only.
- Resize images to 224 x 224 pixels and normalize them using the computed statistics.
- Create an equal random partition of the training set across three clients. Initialize the selected pretrained CNN and replace its final classification layer with the required number of crop classes.
- For each communication round, reset each local model to the current global state and train it for one local epoch using SGD.
- Aggregate the three local model states using weighted FedAvg.
- Evaluate the aggregated global model on the validation split and record Accuracy, weighted F1-score, and round time.
- Repeat for 30 rounds and save per-round results and the final summary.

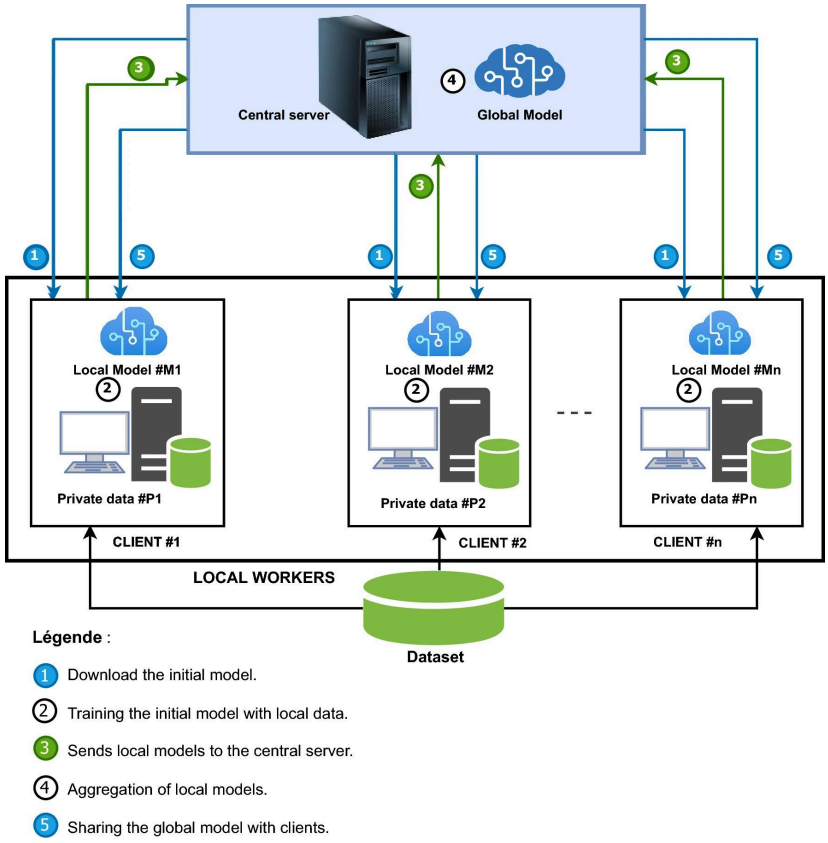


Figure 1. Federated learning client-server workflow showing local training, model update transfer, server-side aggregation, and global model distribution.

4.2 Mathematical Formulation of FedAvg

Let W_t denote the global model parameters at communication round t. Client k trains on n_k local samples and produces W_(t+1)^k. The weighted FedAvg aggregation used in the notebook is:

$$W_{(t+1)} = \sum_k \left(n_k / \sum_j n_j\right) W_{(t+1)}^k$$

Because the client partitions are approximately equal in size, the weights are also approximately equal. The implementation nevertheless computes the exact sample-weighted aggregation for each experiment.

4.3 Client Update

The local optimizer is stochastic gradient descent (SGD) with learning rate 0.01. The loss function is crossentropy. For each client and each communication round, the current global state is loaded into the local model before local optimization. The resulting local state is then returned to the server for aggregation.

4.4 Reproducibility Controls

The notebook sets the random seed to 42 for Python, NumPy, PyTorch, and CUDA, and enables deterministic cuDNN behavior. The experiment outputs include a per-round CSV history and a final summary CSV for every dataset-model pair. This makes it possible to recreate the analysis without retraining the models as long as the saved experiment files are retained.

5. DATASET DESCRIPTION AND PREPROCESSING

5.1 PlantVillage Subsets

The reference paper uses the publicly available PlantVillage dataset and selects four plant types for experimentation: grape, apple, corn, and tomato. The internship implementation uses the same four crop categories, with the class folders corresponding to healthy leaves and specific disease conditions.

Dataset	Classes	Train	Validation	Total	Class categories
Grape	4	3,251	811	4,062	Black rot; Esca (Black Measles); Leaf blight; Healthy
Apple	4	2,536	635	3,171	Apple scab; Black rot; Cedar apple rust; Healthy
Corn	4	3,080	772	3,852	Cercospora leaf spot/Gray leaf spot; Common rust; Northern leaf blight; Healthy

Dataset	Classes	Train	Validation	Total	Class categories
Tomato	10	14,529	3,631	18,160	Bacterial spot; Early blight; Late blight; Leaf Mold; Septoria leaf spot; Spider mites; Target Spot; Yellow Leaf Curl Virus; MosaicVirus; Healthy

Table 1. Dataset composition used in the project.

The implementation contains a small data-count discrepancy for Apple: the final local dataset contains 3,171 images, whereas the reference paper reports 3,161 images. The project retains the actual local dataset count rather than altering the experimental data to match the published total.

5.2 Preprocessing

- Images are converted to RGB.
- All images are resized to 224 x 224 pixels.
- Training-set mean and standard deviation are computed before normalization.
- The same normalization parameters are applied to both training and validation images.
- No additional augmentation is used in the final notebook.

Computing normalization statistics from the training images only avoids using validation information to define the transformation parameters.

6. MODEL ARCHITECTURES

6.1 ResNet50

ResNet50 is a 50-layer convolutional architecture built around residual connections. In the implementation, the pretrained model is loaded from torchvision and its final fully connected

layer is replaced with a linear layer having the number of classes required by the current crop dataset.

6.2 VGG16

VGG16 is a deep convolutional architecture composed of stacked convolutional blocks followed by a classifier. The implementation loads pretrained torchvision weights and replaces the final classifier layer so that it produces four logits for Grape, Apple, or Corn and ten logits for Tomato.

6.3 MobileNetV2

MobileNetV2 is a lightweight CNN architecture designed for efficient computation. In this project, the final classifier is replaced with a task-specific linear layer. The model is evaluated under exactly the same federated configuration as ResNet50 and VGG16.

6.4 Model Selection

The reference paper studies a wider collection of architectures, including DenseNet121, InceptionV3, and Vision Transformer variants. The internship project deliberately narrows the final experiment set to three CNN models so that all four crop datasets can be evaluated consistently under the selected 30-round federated configuration.

7. FEDERATED LEARNING IMPLEMENTATION

7.1 Client Simulation

The notebook simulates three clients inside one Python process. The training samples for a crop are randomly partitioned into three approximately equal subsets. Each client has its own DataLoader over its assigned subset, while the raw images remain in the local logical partition during training.

7.2 Server Aggregation

At the start of a round, the current global state is copied. Each client receives this state and performs one local epoch. The three resulting states are then combined using weighted FedAvg. The new global model is evaluated before the next round begins.

7.3 Evaluation

After every communication round, the global model is evaluated using the validation DataLoader. The implementation computes overall Accuracy and weighted F1-score using scikit-learn. The F1 implementation uses weighted averaging over the crop classes. There is no separate test split in the final notebook.

7.4 Result Storage

For each dataset-model pair, the notebook saves the per-round history containing round number, Accuracy, F1-score, and round execution time. It also saves a summary containing dataset, model, class count, total communication rounds, final Accuracy, and final F1-score. These files are then reused to create the report graphs without retraining.

8. EXPERIMENTAL SETUP

Parameter	Final value
Datasets	Grape, Apple, Corn, Tomato
Models	ResNet50, VGG16, MobileNetV2
Number of clients	3
Local epochs per round	1
Communication rounds	30
Batch size	16

Learning rate	0.01
Input image size	224 x 224
Optimizer	SGD
Loss function	CrossEntropyLoss
Aggregation	Weighted FedAvg
Random seed	42
Evaluation metrics	Accuracy; weighted F1-score
Compute mode	CUDA-enabled device when available

Table 2. Final experimental configuration.

9. RESULTS AND ANALYSIS

9.1 Overall Experimental Results

Dataset	Model	Classes	Rounds	Accuracy	F1-score
Grape	ResNet50	4	30	99.51%	99.51%
Apple	ResNet50	4	30	100.00%	100.00%
Corn	ResNet50	4	30	97.93%	97.93%
Tomato	ResNet50	10	30	99.48%	99.48%
Grape	VGG16	4	30	99.26%	99.26%
Apple	VGG16	4	30	100.00%	100.00%
Corn	VGG16	4	30	97.80%	97.81%
Tomato	VGG16	10	30	99.53%	99.53%
Grape	MobileNetV2	4	30	99.88%	99.88%

Apple	MobileNetV2	4	30	100.00%	100.00%
Corn	MobileNetV2	4	30	97.02%	97.03%
Tomato	MobileNetV2	10	30	99.20%	99.20%

Table 3. Final results of the 12 experiments.

The 12 experiments produced high final validation performance. The lowest final Accuracy in the set is 97.02% for Corn with MobileNetV2, while the highest recorded Accuracy is 100.00%, achieved by all three models on Apple. The final F1 values closely track Accuracy in all experiments.

Model	**Mean final Accuracy**	**Mean final F1-score**
ResNet50	99.23%	99.23%
VGG16	99.15%	99.15%
MobileNetV2	99.02%	99.03%

Table 4. Mean final performance by model across the four datasets.

9.2 Convergence Behavior

Across the four datasets, Accuracy and F1-score generally increase rapidly during the first several communication rounds and then enter a high-performance plateau. For example, Grape with ResNet50 rises from 92.11% Accuracy in Round 1 to 98.89% by Round 4 and remains near 99.5% thereafter. Apple shows even faster convergence, reaching 99.37% Accuracy with ResNet50 by Round 4 and 100.00% at Round 25 and again at Round 27 onward. Corn converges more gradually and exhibits small fluctuations around the 98% region after the early rounds. Tomato improves from 88.35% in Round 1 with ResNet50 to 99.48% by Round 30.

The round-wise plots below preserve the full 30-point histories recorded during the experiments. These figures are useful for identifying convergence, oscillation, and late-round stabilization rather than relying only on the final summary values.

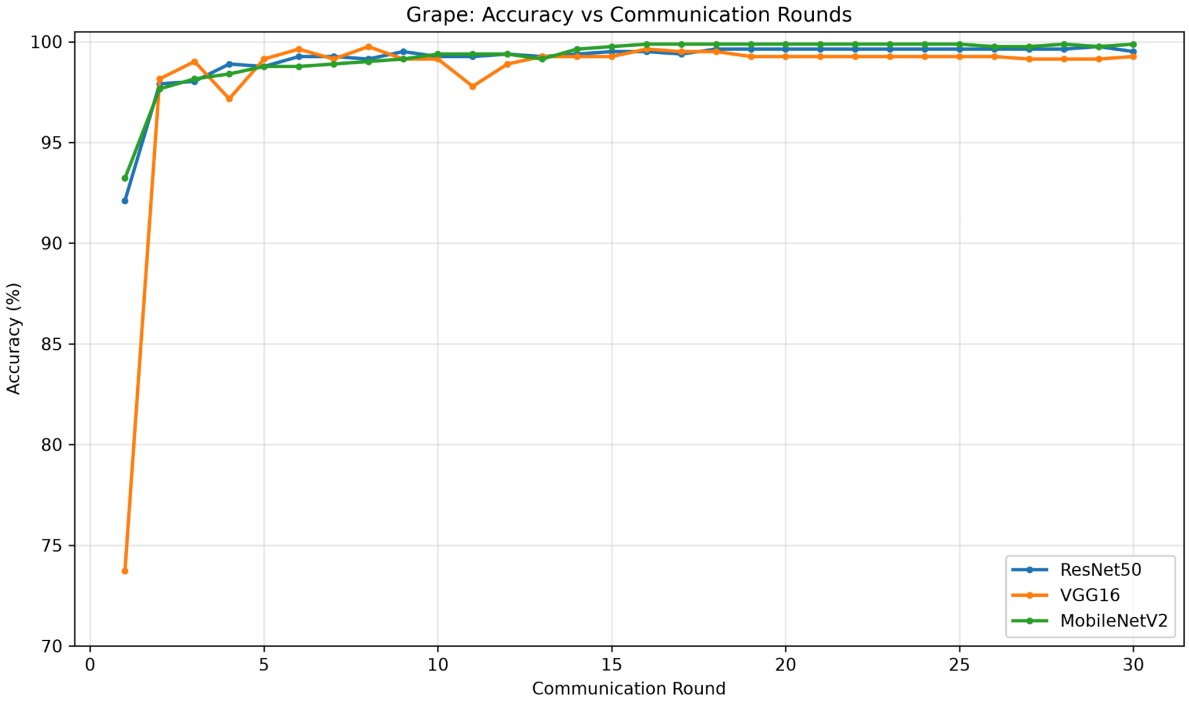


Figure : Accuracy vs communication rounds for Grape (ResNet50, VGG16, MobileNetV2).

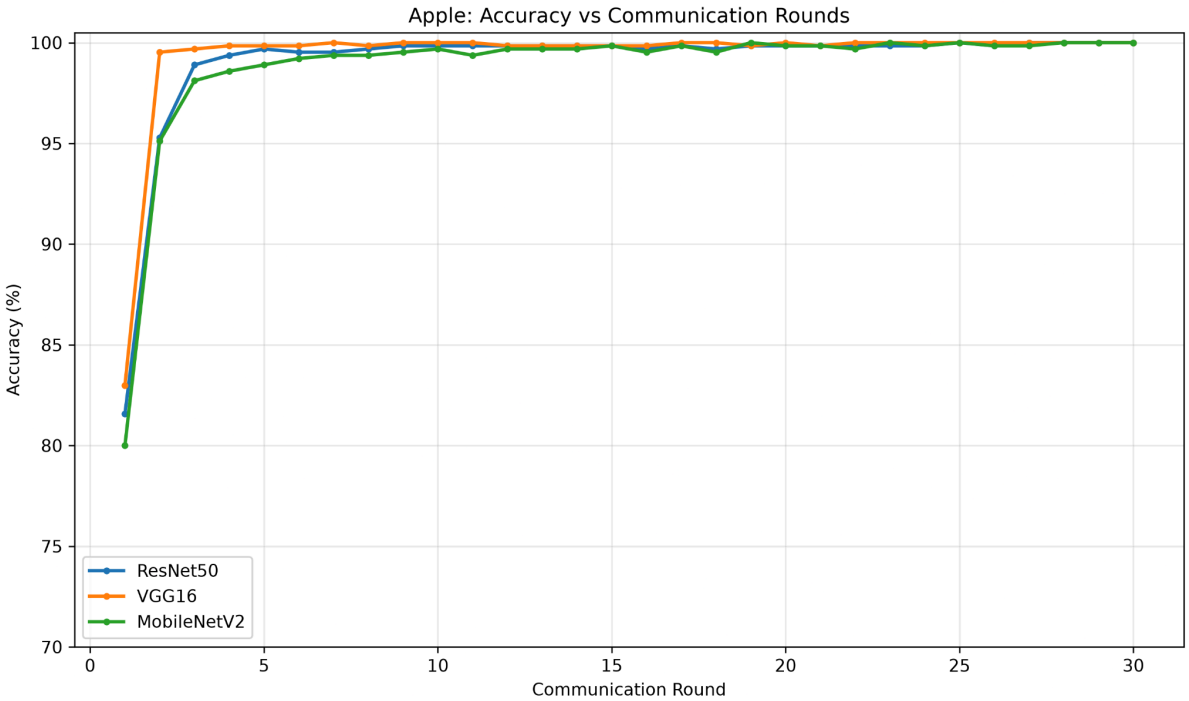


Figure 4: Accuracy vs communication rounds for Apple (ResNet50, VGG16, MobileNetV2).

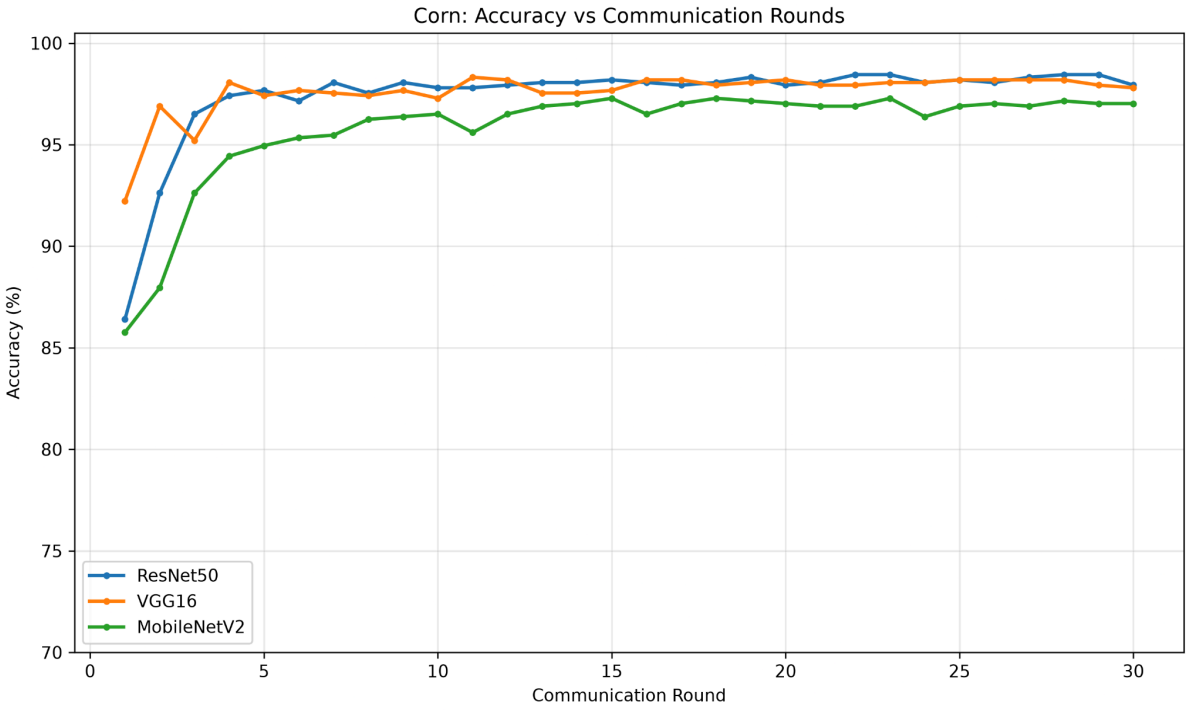


Figure 5: Accuracy vs communication rounds for Corn (ResNet50, VGG16, MobileNetV2).

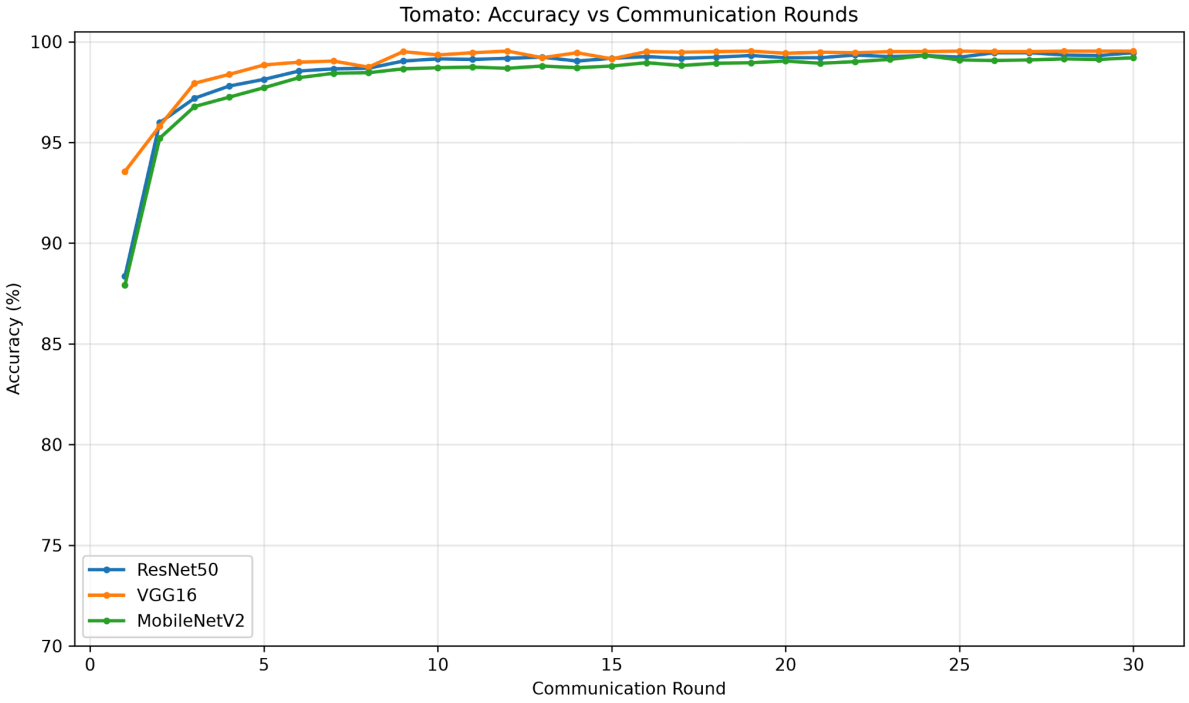


Figure 6: Accuracy vs communication rounds for Tomato (ResNet50, VGG16, MobileNetV2).

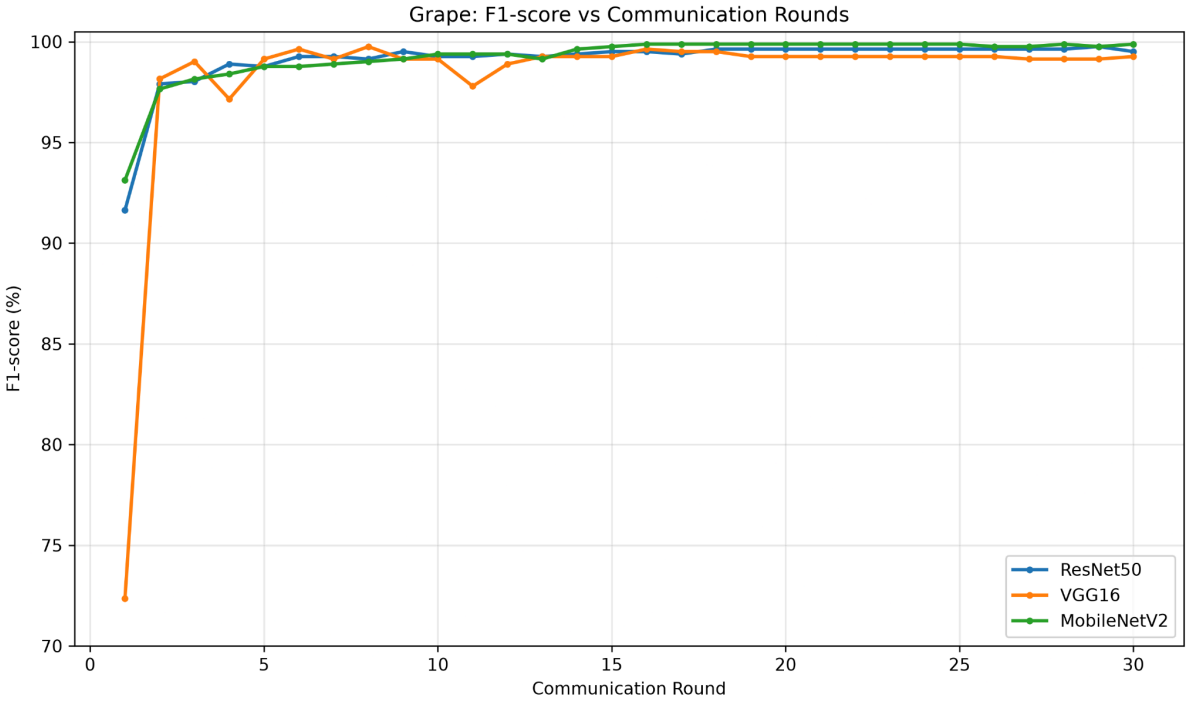


Figure 6: Accuracy vs communication rounds for Grape (ResNet50, VGG16, MobileNetV2).

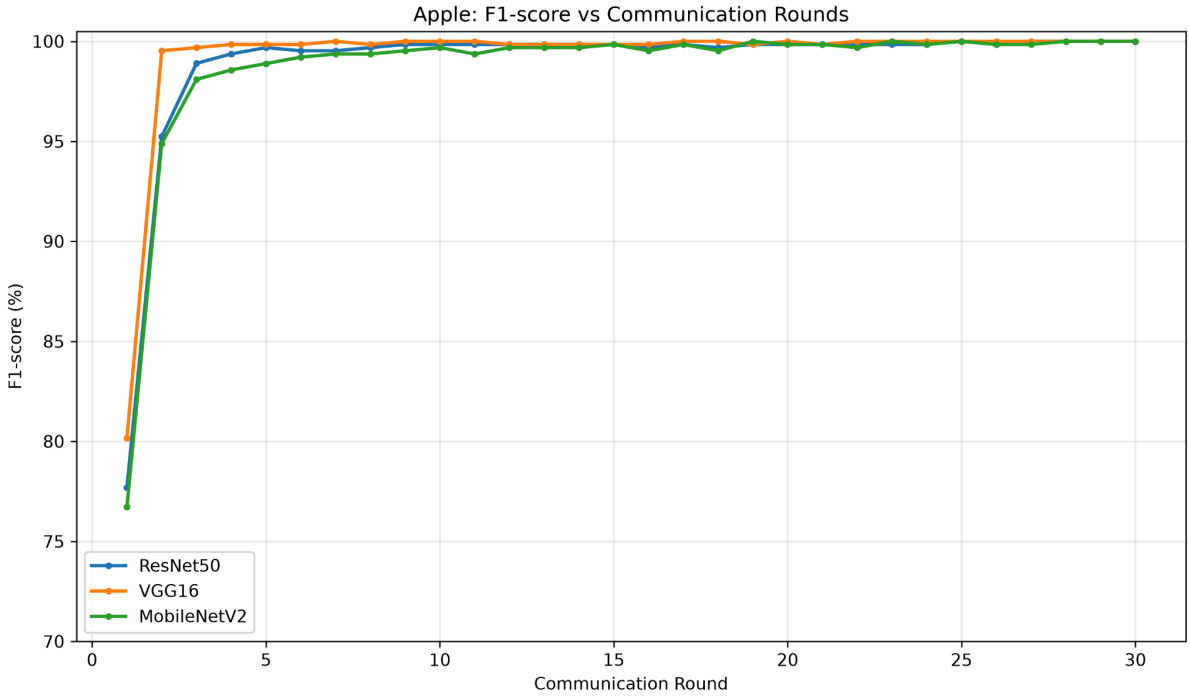


Figure 7: Weighted F1-score vs communication rounds for Apple (ResNet50, VGG16, MobileNetV2).

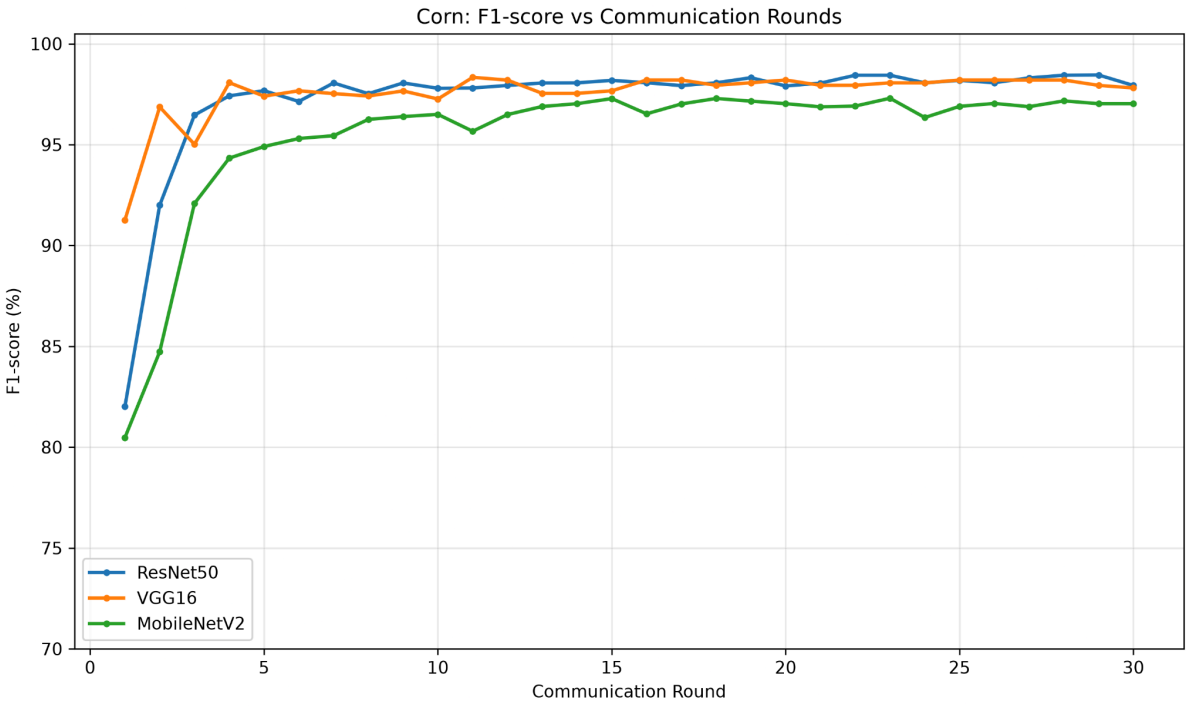


Figure 8: Weighted F1-score vs communication rounds for Corn (ResNet50, VGG16, MobileNetV2).

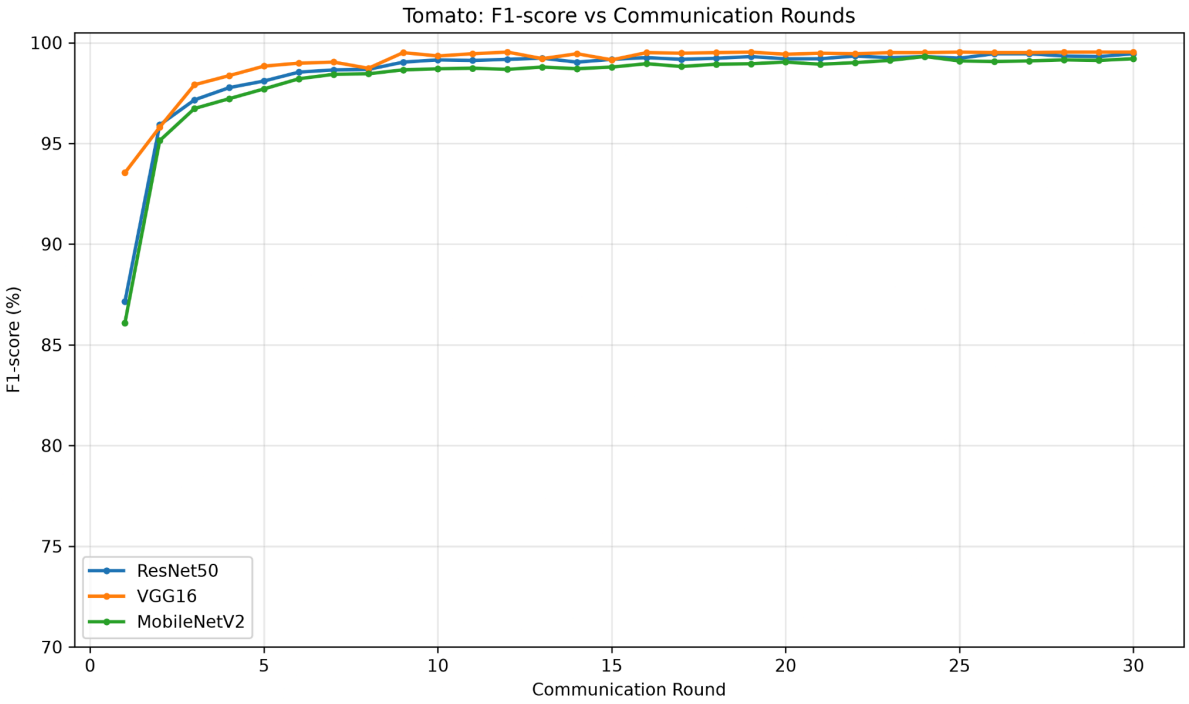


Figure 9: Weighted F1-score vs communication rounds for Tomato (ResNet50, VGG16, MobileNetV2).

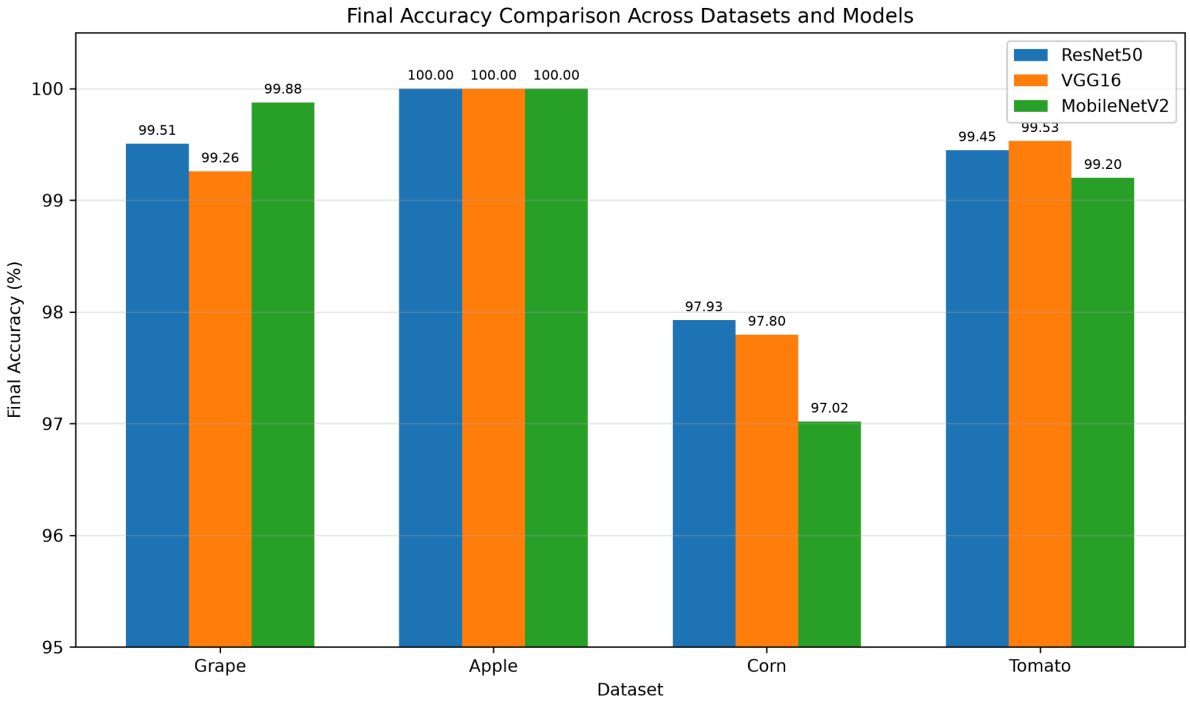


Figure 10. Final Accuracy Comparison Across Datasets and Models.

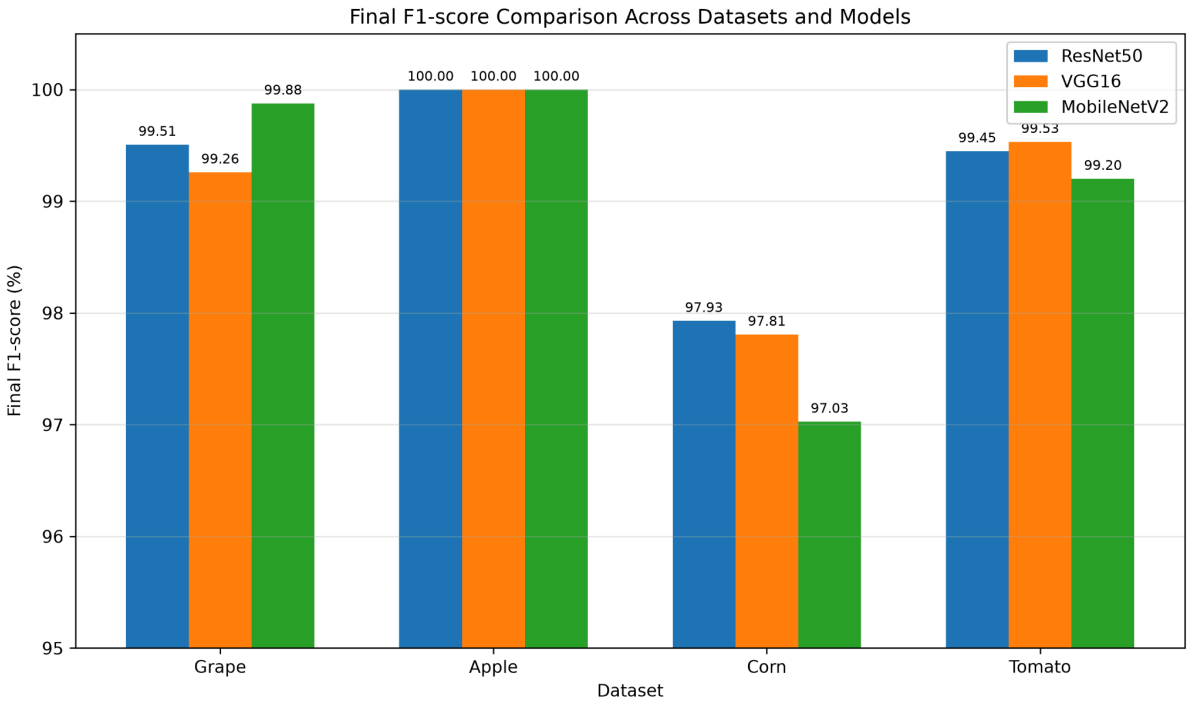


Figure 11. Final Weighted F1-score Comparison Across Datasets and Models.

9.3 Model-wise Summary

The mean final Accuracy across the four datasets is 99.23% for ResNet50, 99.15% for VGG16, and 99.02% for MobileNetV2. The corresponding mean weighted F1-scores are 99.23%, 99.15%, and 99.03%. These averages are descriptive summaries of the four final dataset evaluations; they do not replace the dataset specific results because dataset difficulty and number of classes vary.

9.4 Execution Time

Dataset	Model	Recorded execution time
Grape	ResNet50	17.5 min
Apple	ResNet50	13.6 min
Corn	ResNet50	16.4 min
Tomato	ResNet50	95.1 min
Grape	VGG16	26.9 min
Apple	VGG16	20.7 min
Corn	VGG16	24.7 min
Tomato	VGG16	121.5 min
Grape	MobileNetV2	16.4 min
Apple	MobileNetV2	12.7 min
Corn	MobileNetV2	15.4 min
Tomato	MobileNetV2	103.5 min

Table 5. Execution time by experiment.

The recorded wall-clock times sum to approximately 484.4 minutes, or about 8.07 hours, across the 12 experiments. Tomato experiments require substantially more time because the Tomato

subset is much larger and contains 10 classes. Execution time is therefore an important practical consideration alongside model accuracy.

9.5 Dataset Size

The four datasets contain 4,062 Grape images, 3,171 Apple images, 3,852 Corn images, and 18,160 Tomato images. Tomato therefore dominates the total dataset volume and is also associated with the longest experimental runtimes in the recorded runs.

9.6 Key Findings

The 12 experiments achieved consistently high final validation performance under the fixed 3-client, 30-round, 1-local-epoch configuration. ResNet50 achieved the highest mean final Accuracy (99.23%), followed by VGG16 (99.15%) and MobileNetV2 (99.02%). MobileNetV2 nevertheless remained highly competitive while using a substantially smaller architecture. Apple reached 100.00% final Accuracy with all three models, whereas Corn produced the lowest final Accuracy values. Tomato required the greatest recorded execution time because its dataset is substantially larger. These findings should be interpreted as results on the selected validation splits rather than as evidence of production-level federated privacy or generalization to unseen test data.

10. DISCUSSION

10.1 Interpretation of Performance

The experiments show that the selected CNN architectures can achieve very high validation performance under the fixed 3-client, 30-round, 1-local-epoch setup. The fact that Accuracy and weighted F1-score remain close across all experiments indicates that the aggregate classification performance is consistently high on the selected validation splits.

Apple is the only crop for which all three models reach 100.00% final Accuracy and 100.00% final F1. Grape also produces very high results, with final Accuracy values above 99.2% for all three models. Corn is the most difficult of the four datasets in the final evaluation, with the three models producing approximately 97.0% to 97.9% final Accuracy. Tomato remains above 99.2% for all three models, but requires considerably more computation time because of its larger image count and greater number of classes.

10.2 Relationship to the Reference Paper

The project reproduces the central federated-learning mechanism described by the reference paper, including local training, server-side FedAvg aggregation, repeated communication, and evaluation of the global model. The project does not reproduce every experiment reported in the paper. In particular, the paper varies the number of clients, the number of communication rounds, and local epochs, whereas the final internship implementation fixes these at 3, 30, and 1 respectively.

10.3 Privacy Interpretation

The implementation demonstrates the training mechanics of federated learning by keeping raw training images inside client partitions during the simulated training process. However, because the clients are simulated within a single runtime and the notebook does not implement secure aggregation, differential privacy, encryption, or an actual multi-device deployment, the experiments should be interpreted as a federated-learning simulation rather than a complete production privacy system.

11. LIMITATIONS

- There is no separate held-out test set in the final notebook; the validation folder is used for performance evaluation after each communication round.

- The clients are simulated within one Python process rather than deployed on physically separate devices or machines.
- The client partition is equal and random, so the final study does not evaluate non-IID or highly heterogeneous client data distributions.
- The hyperparameters are fixed rather than optimized separately for each crop or model. Only three of the seven model families discussed in the reference paper are implemented in the final project.
- The final evaluation focuses on overall Accuracy and weighted F1-score; per-class confusion matrices and class-level error analysis were not included in the final experiment logs.
- Communication volume in bytes and network latency were not measured directly; execution time is a local wall-clock proxy rather than a communication-cost measurement.

12. CONCLUSION

This internship project implemented a complete simulated federated learning pipeline for crop disease classification using PlantVillage subsets. The system combines pretrained ResNet50, VGG16, and MobileNetV2 architectures with three simulated clients and weighted FedAvg aggregation. All 12 planned dataset-model combinations were executed for 30 communication rounds with one local epoch per round. The recorded results show high final validation performance across the four crop datasets. Final Accuracy ranges from 97.02% to 100.00%, and the average across the 12 experiments is 99.13%. The experiments also show rapid early-round improvement followed by high-performance stabilization. Execution time varies strongly with dataset size and model architecture, demonstrating that practical federated learning evaluation should consider both predictive performance and computational cost. The main outcome of the internship is therefore a reproducible end-to-end federated image-classification workflow, together with a systematic experimental record covering 12 model-dataset combinations,

30-round convergence histories, final comparisons, execution-time observations, and clearly documented limitations.

13. FUTURE WORK

- Introduce a separate test split so that final model performance is measured on data that is never used for iterative round evaluation.
- Implement non-IID client distributions and compare them with the current equal random partition. Run controlled parameter studies over client count, communication rounds, and local epochs, matching the broader experimental design of the reference paper.
- Add per-class confusion matrices, precision, recall, and classification reports for detailed error analysis. Measure communication payload sizes, round-level communication cost, and end-to-end network latency.
- Add secure aggregation, privacy-preserving mechanisms, or encryption if the project is extended toward a real privacy-aware deployment.
- Evaluate additional architectures such as DenseNet121, InceptionV3, and Vision Transformer variants when computational resources permit.
- Compare federated learning against a centralized-training baseline under a matched data split and preprocessing pipeline.

REFERENCES

[1] D. M. Kabala, A. Hafiane, L. Bobelin, and R. Canals, "Image-based crop disease detection with federated learning," Scientific Reports, vol. 13, art. 19220, 2023. DOI: 10.1038/s41598-023-46218-5.

[2] PlantVillage dataset, as used and described in the reference study for healthy and diseased plant leaf images.

[3] PyTorch and torchvision libraries used for model construction, training, preprocessing, and pretrained model loading in the project notebook.

[4] scikit-learn metrics used for Accuracy and weighted F1-score evaluation in the project notebook.